

0.11.1. Tout l'alphabet, avec textS — de base, ailleurs

Le 0.11 trois lignes plus bas : tout l'alphabet sur les lignes 3 et 4.

1. Ce qu'il faut comprendre

C'est le 0.11, avec un seul changement : la **ligne** : 3 au lieu de 0. `textS` passe tout seul à la ligne 4 après le T.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.11.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <textS>` → [0.10.3. La fonction textS\(\) — un texte qui passe à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **639 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          49 o ← la console, à l'allumage - la mise en route de la console
main()              29 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte         10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles        31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage - vide les deux cartes de l'écran
Tuiles              416 o ← CopierTuiles - 26 lettres sur 43
"ABCDEFGHIIJKLMNO..." 20 o ← texte() - un texte gravé
"UVWXYZ"            6 o ← texte() - un texte gravé

Lettres gravées : ABCDEFGHIJKLMNOPQRSTUVWXYZ
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
texte() × 2        22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1         3 o sur place, dans main() + AttendreImage (28 o)
textS() × 1         0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

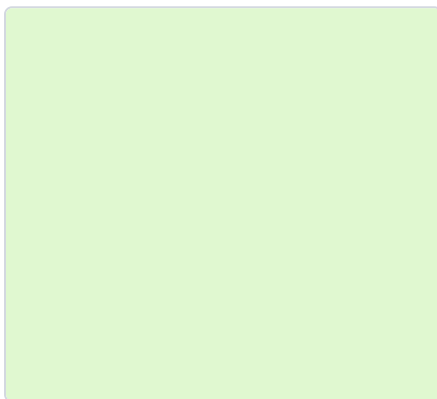
```
#include <textS> // écrit un texte qui passe à la ligne tout seul
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

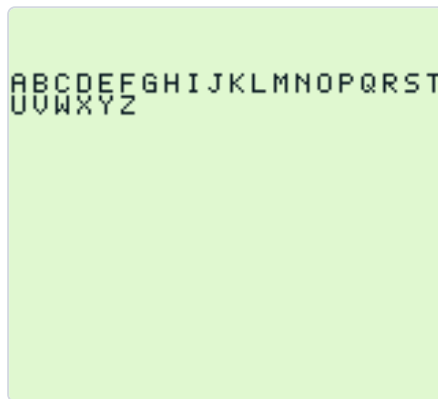
```
int main() {  
    // Le changement : les lignes 0 et 1 → 3 et 4.  
    textS(0, 3, "ABCDEFGH IJKLMNOPQRSTUVWXYZ"); // passe à la ligne 4 après T  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Tout l'alphabet sur les lignes 3 et 4.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ ligne 3 : A à T

✓ ligne 4 : U à Z

Ensuite : **0.11.2. Tout l'alphabet, avec textS — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++